Selective Recovery for Cascading Failures in Multi-Agent LLM Systems

Lucas Yan Atharv Nair Utkarsh Ranjan Ethan Chen Bijin Kurian Jiby

CSE 291A Phase 1 Report (Option 5: Failure Propagation and Recovery in Multi-Agent Systems)

Problem

Multi-agent workflows can fail after completing useful work or causing external side effects. A full restart recomputes every stage, discards valid work, and cannot by itself undo irreversible effects such as sent email. We study recovery after the faulty step is already known: which dependent work must be redone, which work can be reused, and which effects must be undone, compensated, or only reported?

Our current system is a travel-booking pipeline with flight and hotel research stages, a planner, and an executor. Every step records artifacts, dependencies, tool calls, and side effects in a provenance graph. Recovery methods begin from the same faulty world rather than receiving a free reset.

Phase 1 Question

Given a known injected fault, can dependency-aware recovery restore the task while recomputing less work than a full restart?

System and Recovery

The seeded generator creates tasks with exact flight, hotel, constraint, and email ground truth. Ten faults cover observations, agent outputs, plans, tool selection, tool arguments, and one malicious instruction. We compare full restart, suffix replay, coarse lineage, support lineage, monitor, support-monitor, and always-verify. Coarse lineage follows all recorded context dependencies; support lineage follows evidence actually cited by later steps.

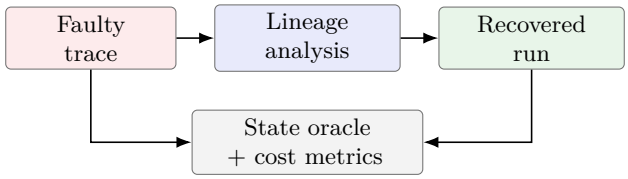

Figure 1: Paired evaluation from one shared faulty execution. Red is the trace, blue is lineage analysis, and green is the recovered run.

Evaluation Design

Five primary metrics answer correctness and cost:

- task recovery success;
- recovery side-effect integrity;
- token ratio relative to restart;
- latency ratio relative to restart; and
- valid-work preservation.

Final-effect integrity, irreversible harm, and global/recovery unauthorized-action rates are reported separately. This prevents damage already caused by the faulty run from being mislabeled as a recovery failure.

Benchmarks and Data

The internal paired harness is the main benchmark because it directly matches our assumption that fault location is known. **UndoBench** adds operational tasks with state and side-effect oracles. Our adapter retries a known pre-mutation failure once; it does not recover a trajectory after a committed effect. **A2ASecBench** adds matched attack/control cases for agent-to-agent protocols. Its adapter measures tainted-action blocking, not rollback after effects occur.

We also reviewed **TAMAS** and **Who&When**. TAMAS requires a paid model API, so it is excluded from Phase 1 numbers without approval. Who&When traces are useful for attribution analysis but are not executable recovery environments for this workflow.

Quantitative Progress

Evidence	Scale	Observed result
Automated suite	64 tests	64 passed
Internal harness	10 faults	mock: all fire; plumbing only
UndoBench smoke	4 pairs	4 controls; 2 recovered
A2ASecBench	800 records	attack success rate 0; control pass 1.0

UndoBench's recovery rate is 0.50 with zero duplicate effects in the four-pair smoke suite. A2ASecBench covers ASRF, ATSI, coordination oscillation, and task flooding (200 records each; one trial per suite). Its perfect preliminary score is policy-specific: the task-flooding adapter uses a declared five-task quota and still needs a sensitivity sweep.

Phase 1 Status

The repository meets the Phase 1 requirement. It contains a reproducible dataset generator, ten fault injections, explicit oracles and metrics, seven recovery/prevention methods, two executable external adapters, machine-readable result rows, provenance traces, and 64 passing tests.

The final empirical claim is not complete. Phase 2 must run the fixed local Qwen2.5-7B model on 30 seeds, 10 faults, and six recovery methods (1,800 paired comparisons), retain failures and timeouts, repeat runs for confidence intervals, evaluate held-out seeds, and sweep the task-flooding quota. Mock-LLM and unapproved paid-API outputs will not be reported as results.

Appendix A: Timeline and Milestones

Period	Milestone and deliverable	Exit criterion
Phase 1: Harness	Seeded multi-agent workflow, provenance graph, effect classes, deterministic world oracle, and ten fault injections.	**Complete**
Phase 1: Evaluation	Five primary metrics; separate final/recovery effects and irreversible harm; JSONL summaries and trace artifacts.	**Complete**
Phase 1: External validation	Native UndoBench and A2ASecBench adapters, automated regression tests, and reproduced smoke results.	**Complete**
Phase 2, Week 1	Measure local-model clean-run reliability and determinism; freeze model, prompts, temperature, seeds, and schedules.	Reliable clean baseline
Phase 2, Week 2	Run 30 seeds $\times$ 10 faults $\times$ 6 recovery methods; retain failures and compute paired token/latency ratios.	1,800 paired trials
Phase 2, Week 3	Repeat runs, calculate confidence intervals, evaluate held-out seeds, and sweep task quotas (1, 5, 10, 20).	Stable estimates
Final phase	Analyze failure cases and ablations; verify claims against artifacts; produce final tables, report, and presentation.	Submission package

Appendix B: Reproducibility

Main paired experiment

```
uv sync
uv run pytest -q
# In a separate terminal:
./scripts/start_server.sh
# Back in this terminal:
uv run agentrec experiment \
  --tasks 30 --faults all \
  --methods restart,suffix,coarse,\
support,monitor,support_monitor
uv run python analysis/summarize.py \
  results/experiment-server-*/rows.jsonl
```

External benchmark smoke tests

```
uv run agentrec undobench download
uv pip install -e data/undobench-upstream
uv run agentrec undobench trial \
  --suite smoke --recovery agentrec

uv run agentrec a2a download
uv pip install -e \
  'data/a2asec-upstream[dev]'
for s in asrf atsi co hotf; do
  uv run agentrec a2a trial \
    --config configs/offline/${s}_eval.yaml \
    --agentrec
done
```